实验 4：数的机器级表示

1. 实验目的

(1)掌握 gcc 的用法;
(2)了解目标代码、库函数的使用;
(3)掌握静态库和共享库的构造与使用;

2. 实现设备

一台装有 Windows 操作系统和 Linux 机系统的微机或服务器。

3. 实验方法与注意事项

实验室内的实验环境与系统是共用设施，请不要在系统内做对系统或对其他用户不安全的事情。
要求每个同学登录后系统后，要在自己的家目录内容以自己名字和学号，创建一个子目录（已有者可以不再创建）。以后所有工作都要在自己的目录内进行。建议以后的实验都在同台计算机上做，这样可以保持连续性。
1)由于是以 root 用户工作的，权力很大，请不要在系统内做对系统或对其他用户不安全的事情。
2)要求每个同学登录后系统后，要在自己的家目录内容以自己(汉语拼音)名字，使用 mkdir 命令创建一个子目录。以后所有工作都要在自己的目录内进行。
3）认真编写实验报告并上传服务器。
用户要按通常实验要认真书写实验报告。

4. 实验过程

（1）欧几里得算法程序（C 代码与汇编代码对比）

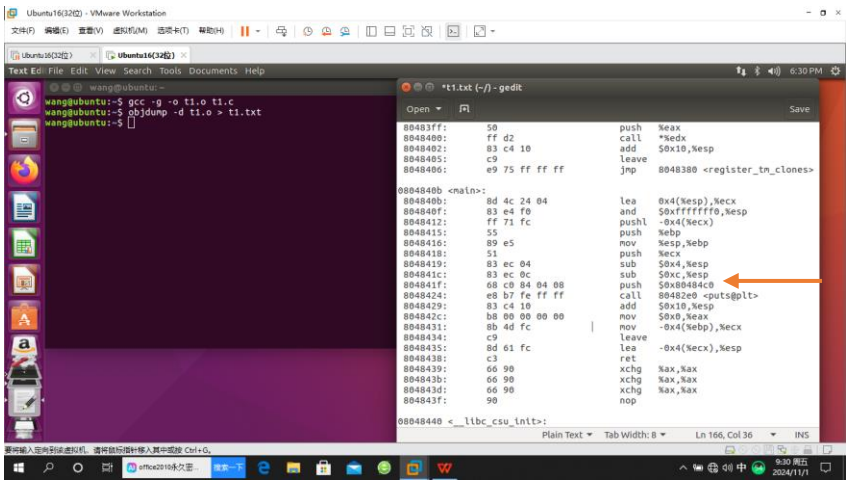
```c
#include<stdio.h>
void main()
{
    int temp;
    int a,b;
    scanf("%d",&a);
    scanf("%d",&b);
    printf("the greatest common factor of %d and %d is ",a,b);
    while(b!=0)
    {
        temp=b;
        b=a%b;
        a=temp;
    }
    printf("%d\n",a);
    getchar();
    getchar();
}
```

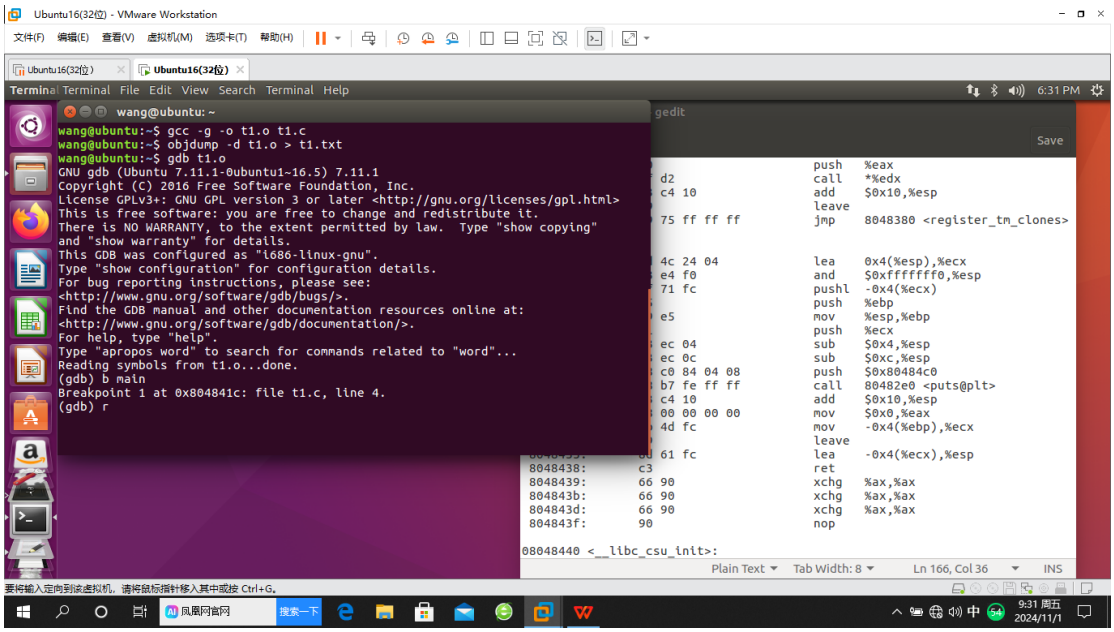
```
wang@ubuntu:~$ gcc -g -o t2.o t2.c
wang@ubuntu:~$ ./t2
bash: ./t2: No such file or directory
wang@ubuntu:~$ ./t2.o
12
24
the greatest common factor of 12 and 24 is 12

wang@ubuntu:~$
```

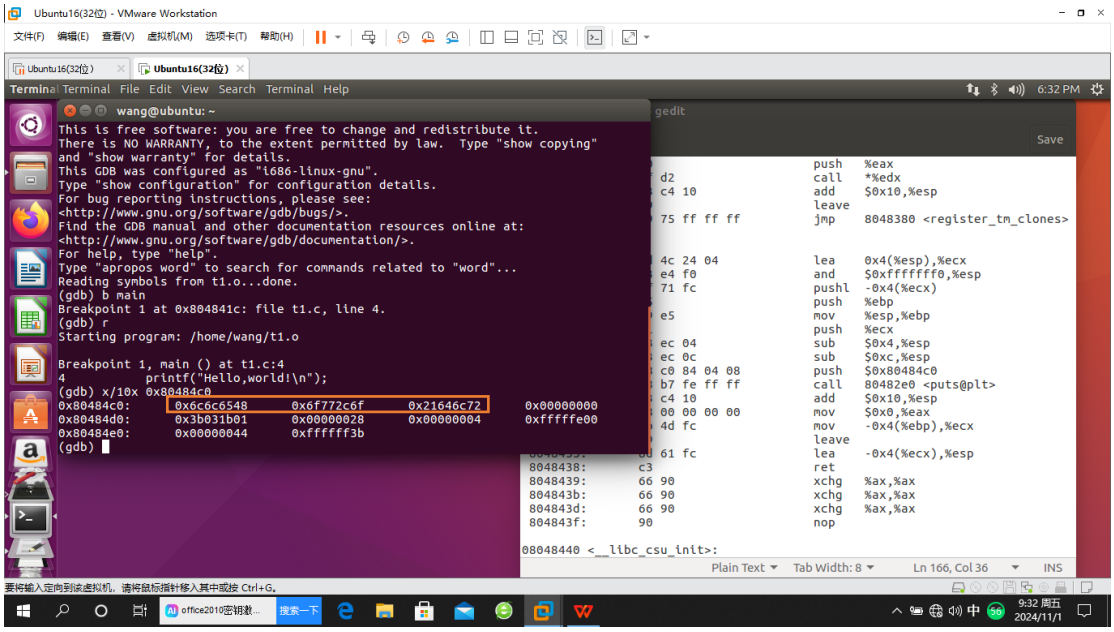
```
080484eb <main>:
 80484eb:   8d 4c 24 04          lea    0x4(%esp),%ecx
 80484ef:   83 e4 f0             and    $0xfffffff0,%esp
 80484f2:   ff 71 fc             pushl  -0x4(%ecx)
 80484f5:   55                   push   %ebp
 80484f6:   89 e5                mov    %esp,%ebp
 80484f8:   51                   push   %ecx
 80484f9:   83 ec 14             sub    $0x14,%esp
 80484fc:   65 a1 14 00 00 00    mov    %gs:0x14,%eax
 8048502:   89 45 f4             mov    %eax,-0xc(%ebp)
 8048505:   31 c0                xor    %eax,%eax
 8048507:   83 ec 08             sub    $0x8,%esp
 804850a:   8d 45 e8             lea    -0x18(%ebp),%eax
 804850d:   50                   push   %eax
 804850e:   68 30 86 04 08       push   $0x8048630
 8048513:   e8 b8 fe ff ff       call   80483d0 <__isoc99_scanf@plt>
 8048518:   83 c4 10             add    $0x10,%esp
 804851b:   83 ec 08             sub    $0x8,%esp
 804851e:   8d 45 ec             lea    -0x14(%ebp),%eax
 8048521:   50                   push   %eax
 8048522:   68 30 86 04 08       push   $0x8048630
 8048527:   e8 a4 fe ff ff       call   80483d0 <__isoc99_scanf@plt>
 804852c:   83 c4 10             add    $0x10,%esp
 804852f:   8b 55 ec             mov    -0x14(%ebp),%edx
 8048532:   8b 45 e8             mov    -0x18(%ebp),%eax
 8048535:   83 ec 04             sub    $0x4,%esp
 8048538:   52                   push   %edx
 8048539:   50                   push   %eax
 804853a:   68 34 86 04 08       push   $0x8048634
 804853f:   e8 4c fe ff ff       call   8048390 <printf@plt>
 8048544:   83 c4 10             add    $0x10,%esp
 8048547:   eb 1a                jmp    8048563 <main+0x78>
 8048549:   8b 45 ec             mov    -0x14(%ebp),%eax
 804854c:   89 45 f0             mov    %eax,-0x10(%ebp)
 804854f:   8b 45 e8             mov    -0x18(%ebp),%eax
 8048552:   8b 4d ec             mov    -0x14(%ebp),%ecx
 8048555:   99                   cltd
 8048556:   f7 f9                idiv   %ecx
 8048558:   89 d0                mov    %edx,%eax
 804855a:   89 45 ec             mov    %eax,-0x14(%ebp)
 804855d:   8b 45 f0             mov    -0x10(%ebp),%eax
 8048560:   89 45 e8             mov    %eax,-0x18(%ebp)
 8048563:   8b 45 ec             mov    -0x14(%ebp),%eax
 8048566:   85 c0                test   %eax,%eax
 8048568:   75 df                jne    8048549 <main+0x5e>
 804856a:   8b 45 e8             mov    -0x18(%ebp),%eax
 804856d:   83 ec 08             sub    $0x8,%esp
 8048570:   50                   push   %eax
 8048571:   68 60 86 04 08       push   $0x8048660
 8048576:   e8 15 fe ff ff       call   8048390 <printf@plt>
 804857b:   83 c4 10             add    $0x10,%esp
 804857e:   e8 1d fe ff ff       call   80483a0 <getchar@plt>
 8048583:   e8 18 fe ff ff       call   80483a0 <getchar@plt>
 8048588:   90                   nop
 8048589:   8b 45 f4             mov    -0xc(%ebp),%eax
 804858c:   65 33 05 14 00 00 00 xor    %gs:0x14,%eax
 8048593:   74 05                je     804859a <main+0xaf>
 8048595:   e8 16 fe ff ff       call   80483b0 <__stack_chk_fail@plt>
 804859a:   8b 4d fc             mov    -0x4(%ebp),%ecx
 804859d:   c9                   leave
 804859e:   8d 61 fc             lea    -0x4(%ecx),%esp
 80485a1:   c3                   ret
```

（2）反汇编如下程序，并找出“Hello,World”的位置。

```c
#include <stdio.h>
int main ()
{
    printf("Hello, world!\n");
}
```


Ubuntu16(32位) - VMware Workstation
wang@ubuntu:~$ gcc -g -o t1.o t1.c
wang@ubuntu:~$ objdump -d t1.o > t1.txt
wang@ubuntu:~$
*t1.txt (~/) - gedit
08048440 <__libc_csu_init>:


Ubuntu16(32位) - VMware Workstation
wang@ubuntu: ~
wang@ubuntu:~$ gcc -g -o t1.o t1.c
wang@ubuntu:~$ objdump -d t1.o > t1.txt
wang@ubuntu:~$ gdb t1.o
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.5) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from t1.o...done.
(gdb) b main
Breakpoint 1 at 0x804841c: file t1.c, line 4.
(gdb) r


Ubuntu16(32位) - VMware Workstation
wang@ubuntu: ~
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from t1.o...done.
(gdb) b main
Breakpoint 1 at 0x804841c: file t1.c, line 4.
(gdb) r
Starting program: /home/wang/t1.o
Breakpoint 1, main () at t1.c:4
4 printf("Hello,world!\n");
(gdb) x/10x 0x80484c0
0x80484c0: 0x6c6c6548 0x6f772c6f 0x21646c72 0x00000000
0x80484d0: 0x3b031b01 0x00000028 0x00000004 0xffffe00
0x80484e0: 0x00000044 0xffffff3b
(gdb)

（3）

代码段一：

int a = 0x80000000;

int b = a / -1;

printf("%d, %d \n", a, b);

```
wang@ubuntu:~$ gedit t4.c
wang@ubuntu:~$ gcc -g -o t4.o t4.c
wang@ubuntu:~$ ./t4.o
-2147483648,-2147483648
wang@ubuntu:~$
```

代码段二：

int a = 2147483648;

int b = -1;

int c = a / b;

printf("%d, %d\n", a, c);

```
wang@ubuntu:~$ gcc -g -o t41.o t4.c
wang@ubuntu:~$ ./t41.o
Floating point exception (core dumped)
wang@ubuntu:~$
```

为什么两者结果不同！

```
0804840b <main>:
 804840b:       8d 4c 24 04             lea    0x4(%esp),%ecx
 804840f:       83 e4 f0                and    $0xfffffff0,%esp
 8048412:       ff 71 fc                pushl  -0x4(%ecx)
 8048415:       55                      push   %ebp
 8048416:       89 e5                   mov    %esp,%ebp
 8048418:       51                      push   %ecx
 8048419:       83 ec 14                sub    $0x14,%esp
 804841c:       c7 45 f0 00 00 00 80    movl   $0x80000000,-0x10(%ebp)
 8048423:       8b 45 f0                mov    -0x10(%ebp),%eax
 8048426:       f7 d8                   neg    %eax
 8048428:       89 45 f4                mov    %eax,-0xc(%ebp)
 804842b:       83 ec 04                sub    $0x4,%esp
 804842e:       ff 75 f4                pushl  -0xc(%ebp)
 8048431:       ff 75 f0                pushl  -0x10(%ebp)
 8048434:       68 d0 84 04 08          push   $0x80484d0
 8048439:       e8 a2 fe ff ff          call   80482e0 <printf@plt>
 804843e:       83 c4 10                add    $0x10,%esp
 8048441:       b8 00 00 00 00          mov    $0x0,%eax
 8048446:       8b 4d fc                mov    -0x4(%ebp),%ecx
 8048449:       c9                      leave
 804844a:       8d 61 fc                lea    -0x4(%ecx),%esp
 804844d:       c3                      ret
 804844e:       66 90                   xchg   %ax,%ax
```

```
0804840b <main>:
 804840b:       8d 4c 24 04             lea    0x4(%esp),%ecx
 804840f:       83 e4 f0                and    $0xfffffff0,%esp
 8048412:       ff 71 fc                pushl  -0x4(%ecx)
 8048415:       55                      push   %ebp
 8048416:       89 e5                   mov    %esp,%ebp
 8048418:       51                      push   %ecx
 8048419:       83 ec 14                sub    $0x14,%esp
 804841c:       c7 45 ec 00 00 00 80    movl   $0x80000000,-0x14(%ebp)
 8048423:       c7 45 f0 ff ff ff ff    movl   $0xffffffff,-0x10(%ebp)
 804842a:       8b 45 ec                mov    -0x14(%ebp),%eax
 804842d:       99                      cltd
 804842e:       f7 7d f0                idivl  -0x10(%ebp)
 8048431:       89 45 f4                mov    %eax,-0xc(%ebp)
 8048434:       83 ec 04                sub    $0x4,%esp
 8048437:       ff 75 f4                pushl  -0xc(%ebp)
 804843a:       ff 75 ec                pushl  -0x14(%ebp)
 804843d:       68 e0 84 04 08          push   $0x80484e0
 8048442:       e8 99 fe ff ff          call   80482e0 <printf@plt>
 8048447:       83 c4 10                add    $0x10,%esp
 804844a:       b8 00 00 00 00          mov    $0x0,%eax
 804844f:       8b 4d fc                mov    -0x4(%ebp),%ecx
 8048452:       c9                      leave
 8048453:       8d 61 fc                lea    -0x4(%ecx),%esp
 8048456:       c3                      ret
```

用 objdump 看代码段一的反汇编代码，得知除以 -1 被优化成取负指令 neg，故未发生除法溢出.

（4）

int x = –1;

unsigned u = 2147483648;

printf (“x = %u = %d\n”, x, x);

printf (“u = %u = %d\n”, u, u);

在 32 位机器上运行上述代码时，它的输出结果是什么？为什么？

```
wang@ubuntu:~$ gcc -g -o t3.o t3.c
wang@ubuntu:~$ ./t3.o
x = 4294967295 = -1
u = 2147483648 = -2147483648
wang@ubuntu:~$
```

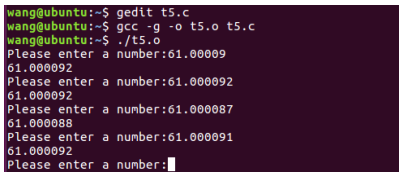
解释：因为–1 的补码整数表示为“11…1”，作为 32 位无符号数解释时，其值为 2^{32}–1= 4 294 967 296–1 = 4 294 967 295。2^{31} 的无符号数表示为“100…0”，被解释为 32 位带符号整数时，其值为最小负数：$-2^{32-1} = -2^{31}$ = –2 147 483 648。

（5） 浮点数的近似表示

int main()

{

```
    Float h;
    While(1){
    Printf("Please enter a number:");
    Scanf("%f",&h);
    Printf("%f\n",h);
    }
}
```



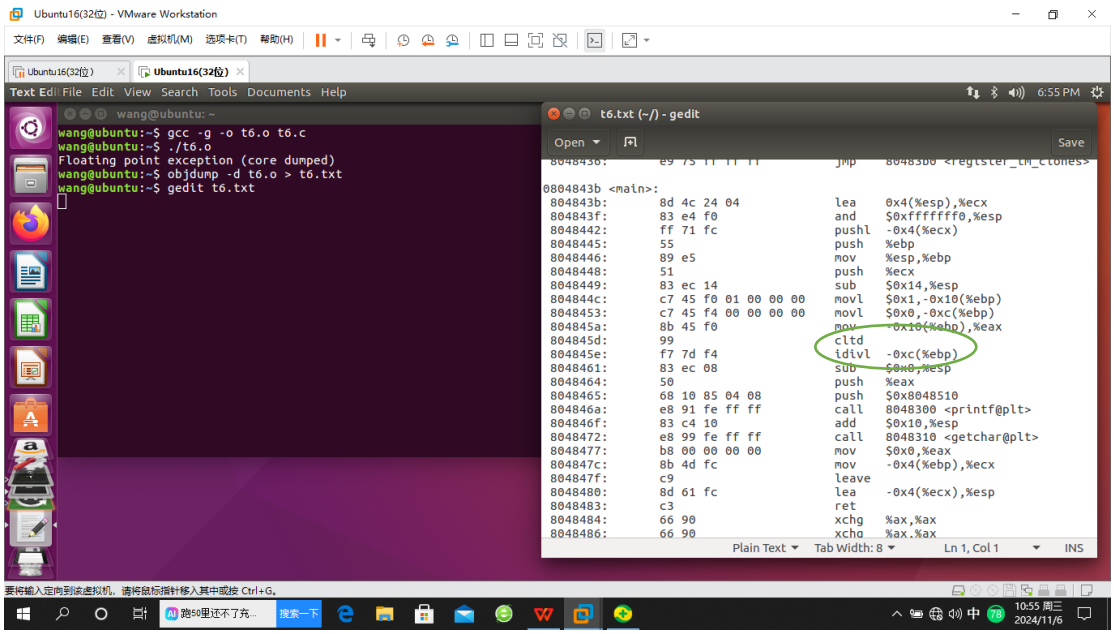
由于计算机位数是固定的，浮点数无法精确表示每一个实数，所以只能进行近似或舍入。

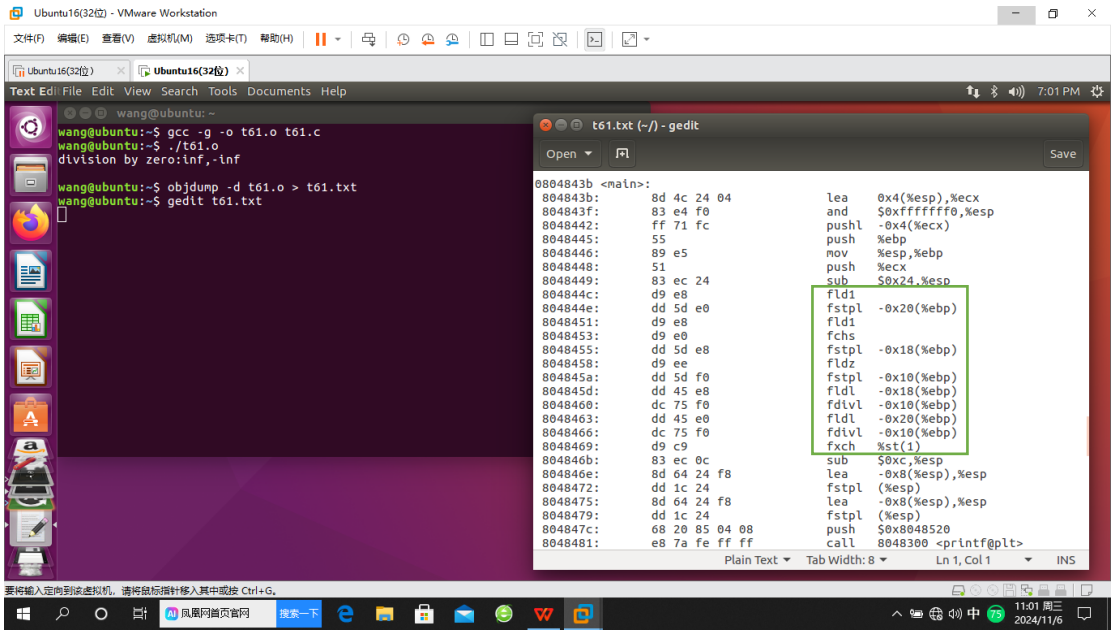
（6）浮点数除 0 问题

代码一：

```
#include <stdio.h>
Int main(){
    Int a=1,b=0;
    Printf("Division by zero:%d\n",a/b);
    Getchar();
}
Int main(){
    Double x=1.0,y=-1.0,z=0.0;
    Printf("division by zero:%f,%f\n",x/z,y/z);
    Getchar();
}
```

问：为什么整除 int 型会产生错误？是什么错误？而用 double 型结果为正负无穷，如何解释？





5. 实验步骤

1. 进入 Linux 系统，在 shell 终端的提示符后输入 gedit，编写 C 语言源程序。
gedit
输入实验内容的源程序并以 sample.c 为文件名将文件存盘。
2. 用 gcc 对源文件 sample.c 进行编译并产生目标文件 sample.o，汇编语言程序 sample.s
gcc -g -o sample.o sample.c
objdump -d sample.o > sample.txt
3. 用 GDB 跟踪执行 sample
gdb sample
4. 观察程序执行情况并记录运行结果。
5. 依照上述 4 个步骤，完成对其余实验内容的操作。

6. 实验任务

1. 判断题:
已知
int x ;
float f ;
double d ;
x == (int)(float) x ( )
x == (int)(double) x ( )
f == (float)(double) f ( )

d == (float) d　　(　　)
f == -(-f);　　(　　)
2/3 == 2/3.0　　(　　)
d * d >= 0.0　　(　　)
x*x>=0　　(　　)
(d+f)-d == f　　(　　)

2. “-2 < 2”和“-2 < 2u”的结果一样吗？为什么？请编写程序验证。
3. 运行下列程序代码，并对程序输出结果进行分析。

```
#include <stdio.h>
void main(){
    unsigned int a = 1;
    unsigned short b = 1;
    char c = -1;
    int d;
    d = (a>c)?1:0;
    printf(“unsigned int is %d\n”,d);
    d = (b>c)?1:0;
    printf(“unsigned short is %d\n”,d);
}
```

4. 运行下列代码，并对输出结果进行分析。

```
#include <stdio.h>
void main(){
    union NUM{
        int a;
        Char b[4];
    }num;
    num.a = 0x12345678;
    printf(“0x%X\n”,num.b[2]);
}
```

4) 请说明下列赋值语句执行后，各个变量对应的机器数和真值各是多少？编写一段程序代码并进行编译，观察默认情况下，编译器是否报 warning？如果有 warning 信息的话，请分析为何会出现这种 warning 信息。

```
int a = 2147483648;
int b = -2147483648;
int c = 2147483649;
unsigned short d = 65539;
Short e = -32790;
```

5) 编译运行以下程序，并至少重复运行 3 次。

```
void main(){
        double x = 23.001, y = 24.001, z = 1.0;
        for (int i=0; i<10; i++){
        if((y-x)==z)
            printf(“equal\n”);
```

```
        else
            printf("not equal\n");
        x+=z;
        y+=z;
        printf("%d,%f,%f\n",i,x,y);
        }
    }
```

要求：

（1） 给出每次运行的结果截图。

（2） 每次运行过程中，是否每一次循环中的判等结果都一致？为什么？

（3） 每次运行过程中，每一次循环输出的 i、x 和 y 的结果分别是什么？为什么？